计算机组成原理
复习

系统总线

- 书后习题3.4、3.7、3.14、3.15、3.16（10分）

3.14 $$\frac{2B}{\frac{1}{8M}} = 16MBps$$

3.15 $$\frac{4B}{4 * \frac{1}{66M}} = 66MBps$$

3.16 波特率：$120 * 12 = 1440bps$

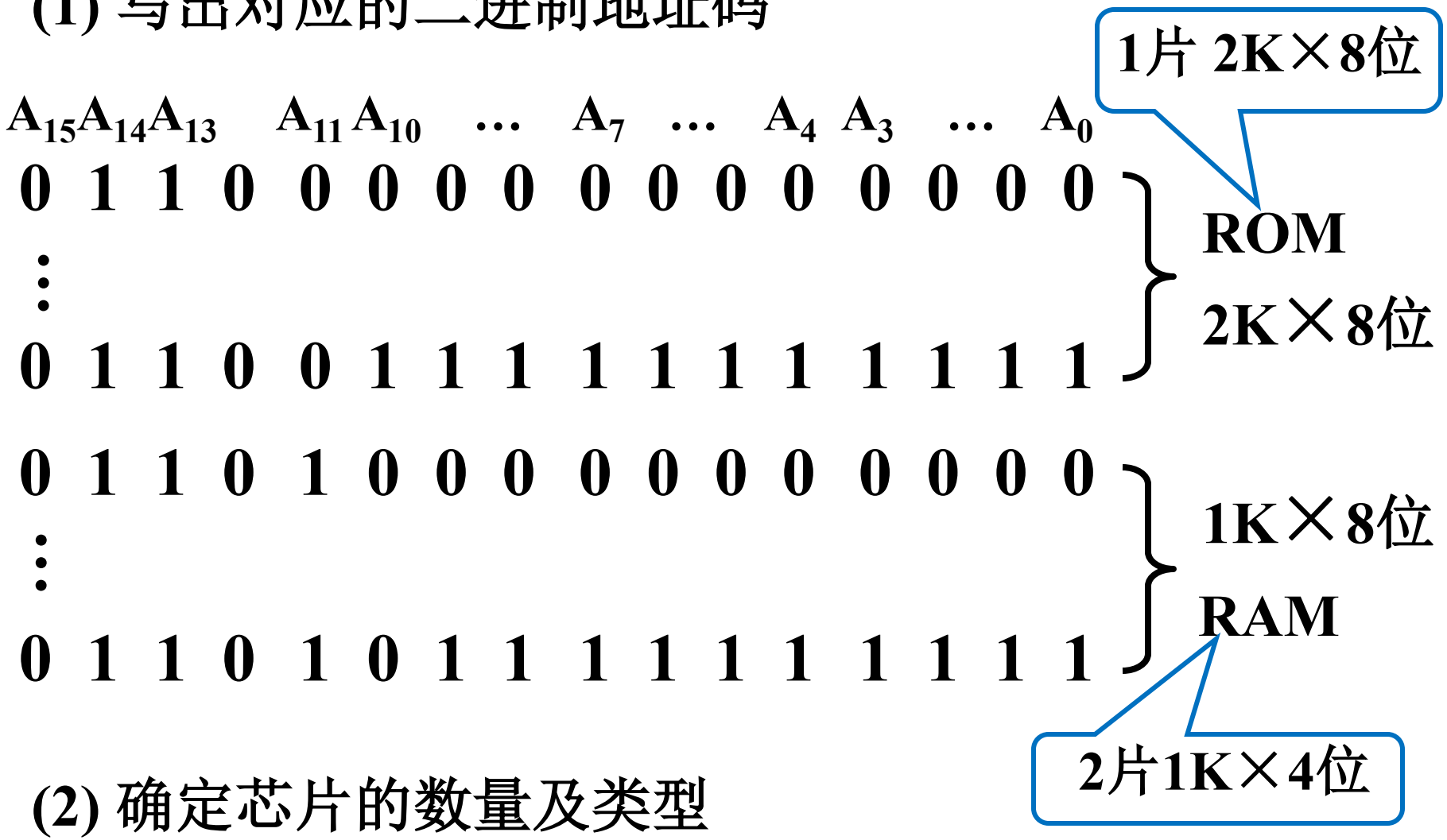
比特率：$120 * 8 = 960bps$

存储器

- 存储器和CPU的连接：例4.1、例4.2、习题4.15、4.16（15分）

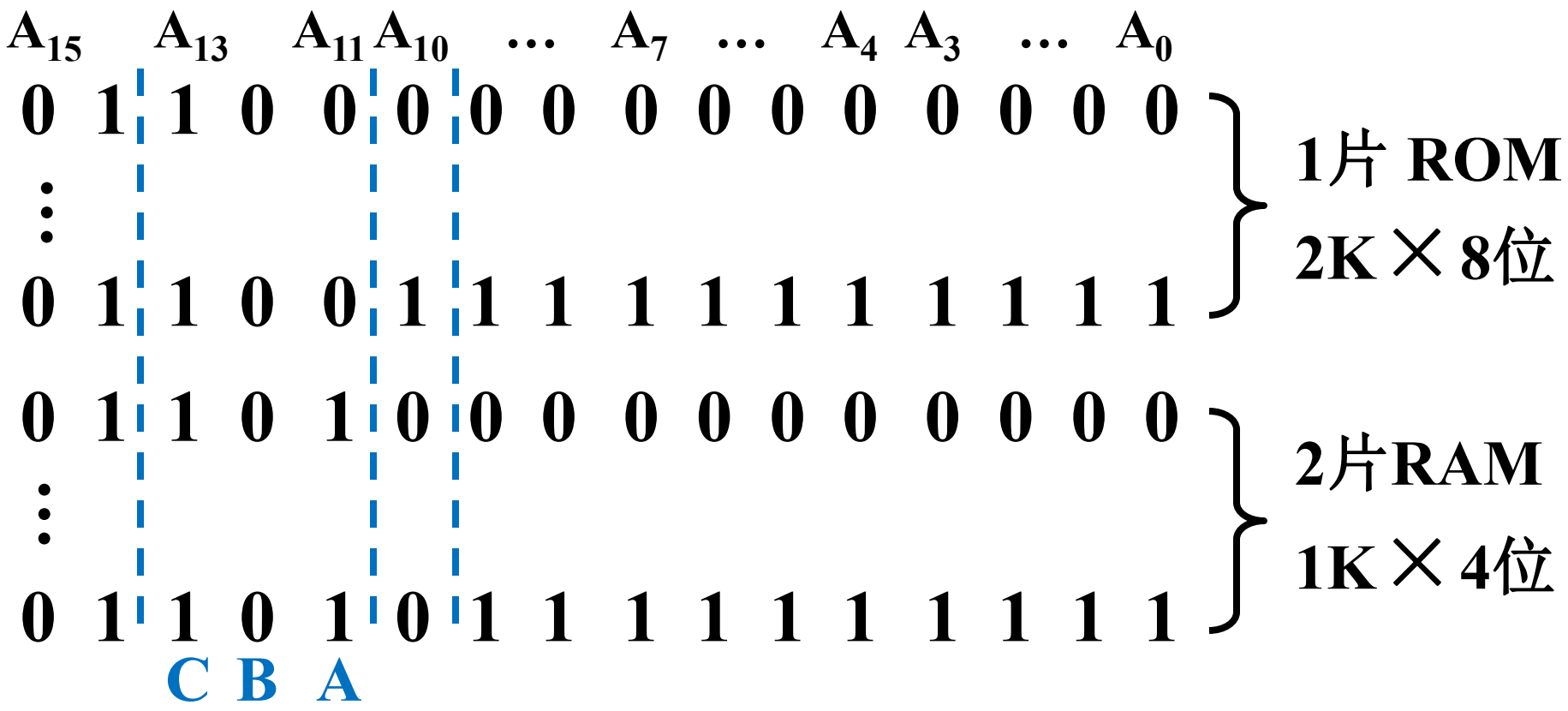
例4.1 解:

(1) 写出对应的二进制地址码



(2) 确定芯片的数量及类型

(3) 分配地址线

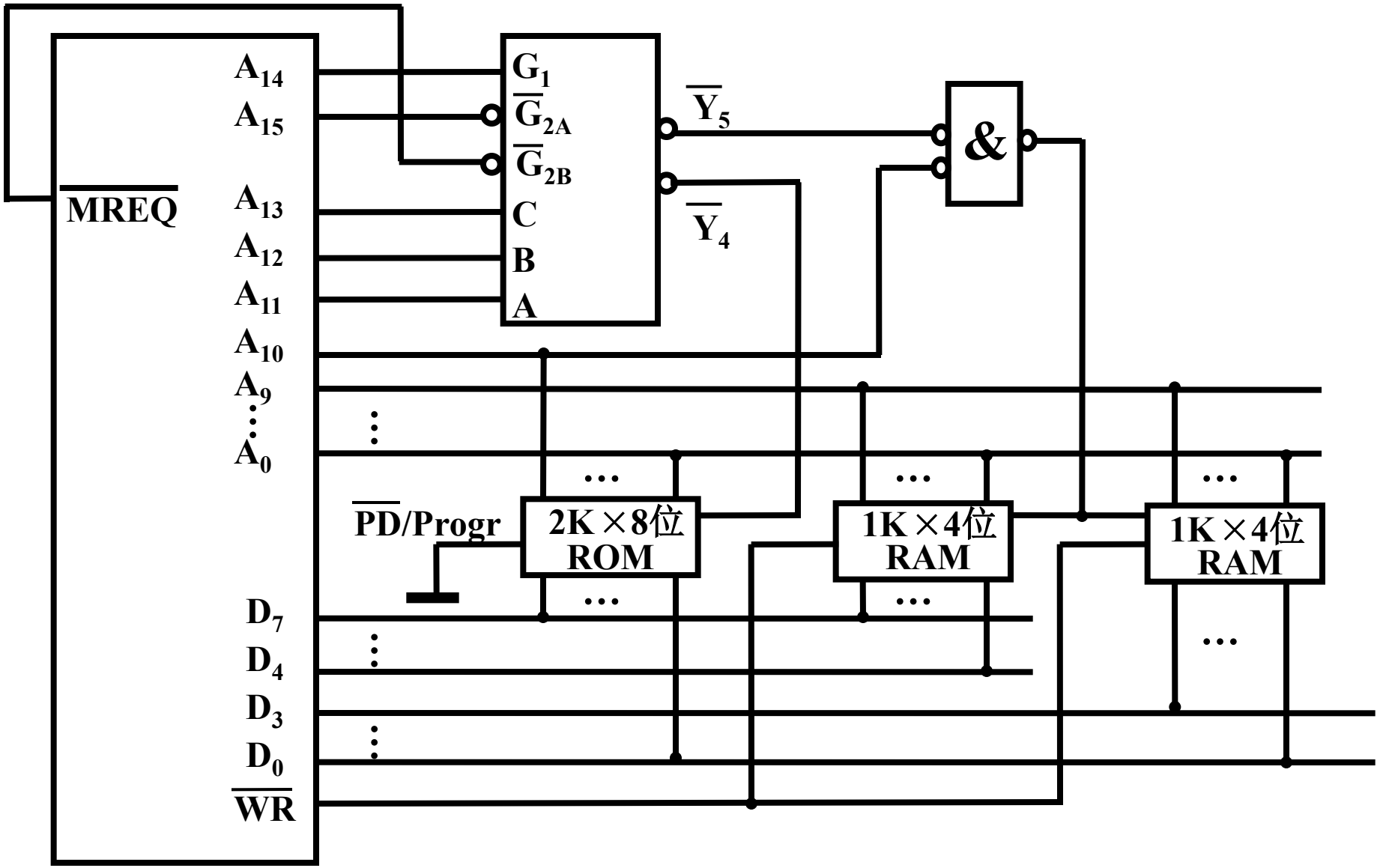


A_{10}~A_0 接 2K×8位 ROM 的地址线

A_9 ~A_0 接 1K×4位 RAM 的地址线

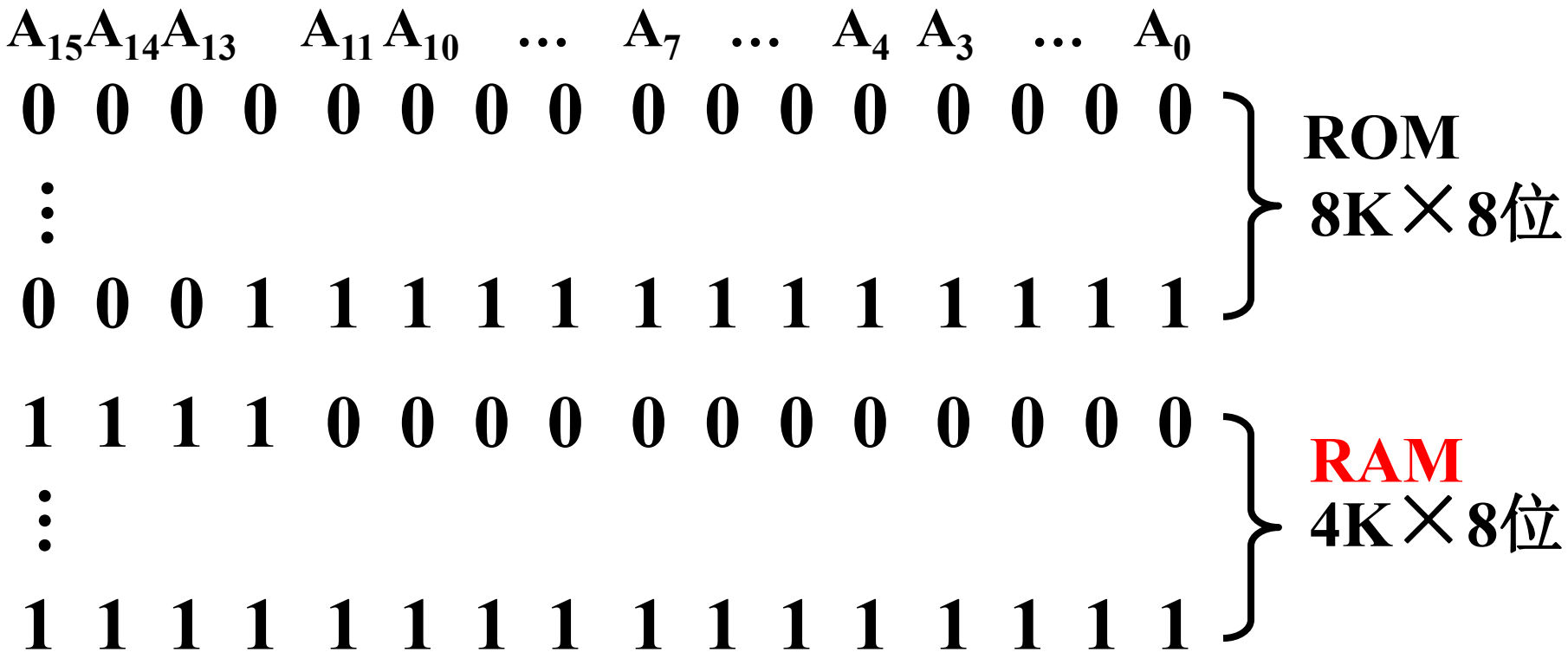
(4) 确定片选信号

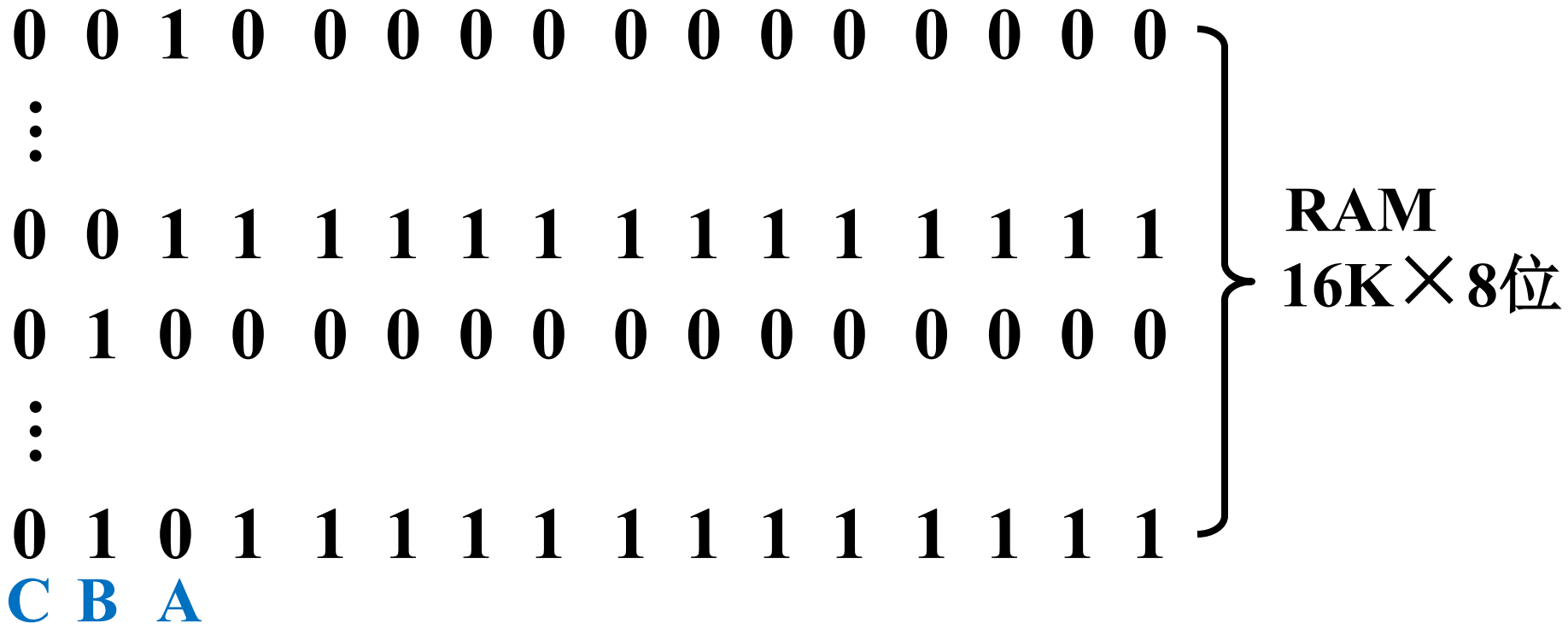
例 4.1 CPU 与存储器的连接图



例4.2 假设同前，要求最小 8K为系统程序区，相邻 16K为用户程序区，最大 4K为系统程序工作区。

(1) 写出对应的二进制地址码





(2) 确定芯片的数量及类型

1片 8K×8位 ROM

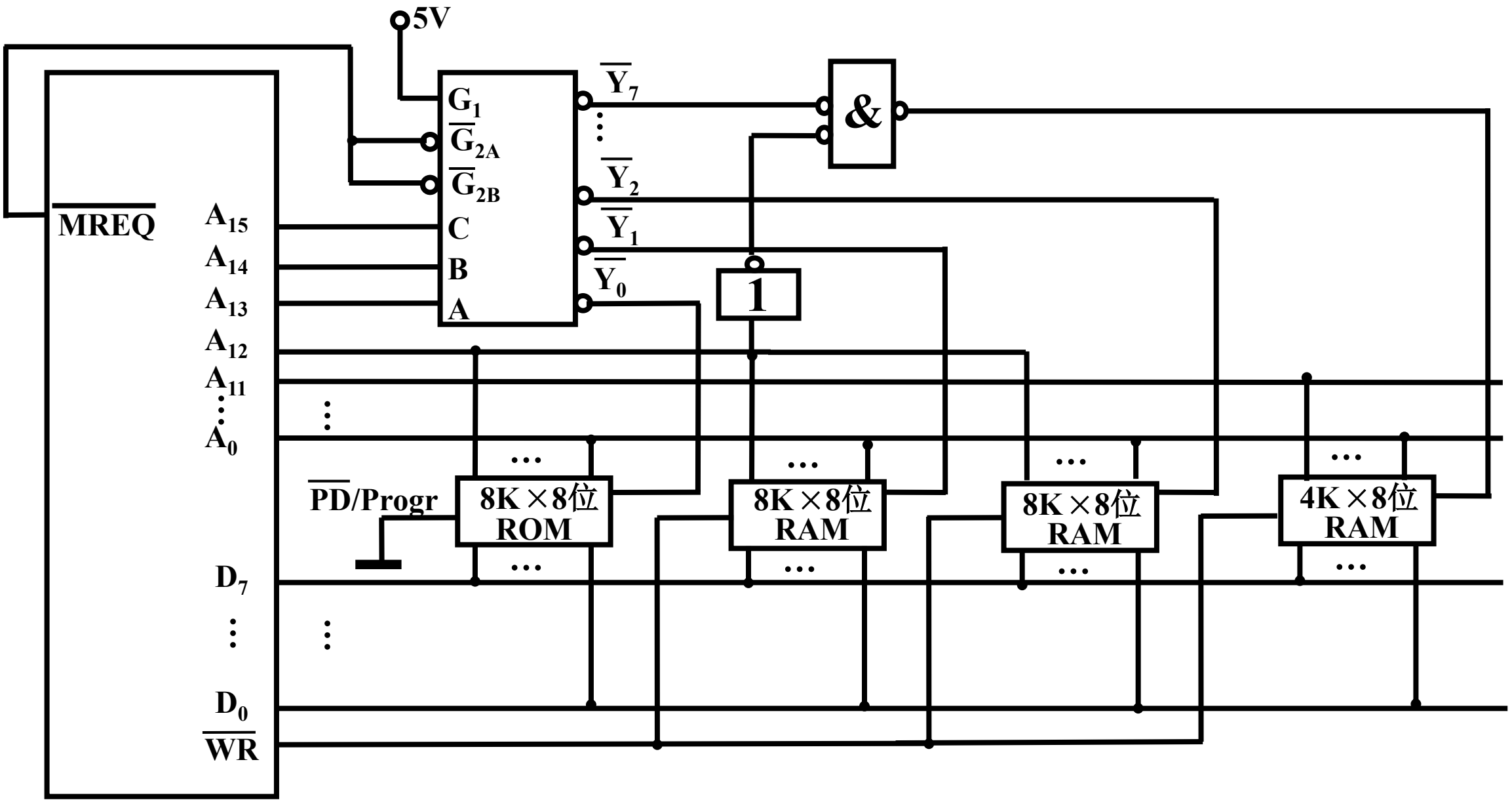
2片 8K×8位 RAM　　1片 4K×8位 RAM

(3) 分配地址线

A_{12}~A_0 接 8K×8位 ROM 和 8K×8位 RAM 的地址线

A_{11}~A_0 接 4K×8位 RAM 的地址线

(4) 确定片选信号


5V
$\overline{MREQ}$
A_{15}
A_{14}
A_{13}
A_{12}
A_{11}
A_0
G_1
$\overline{G}_{2A}$
$\overline{G}_{2B}$
C
B
A
$\overline{Y}_7$
$\overline{Y}_2$
$\overline{Y}_1$
$\overline{Y}_0$
&
1
$\overline{PD}$/Progr
8K×8位 ROM
8K×8位 RAM
8K×8位 RAM
4K×8位 RAM
D_7
D_0
$\overline{WR}$

习题4.15

(1)地址空间分配图:
系统程序区(ROM共4KB):0000H-0FFFH
用户程序区(RAM共12KB):1000H-3FFFH

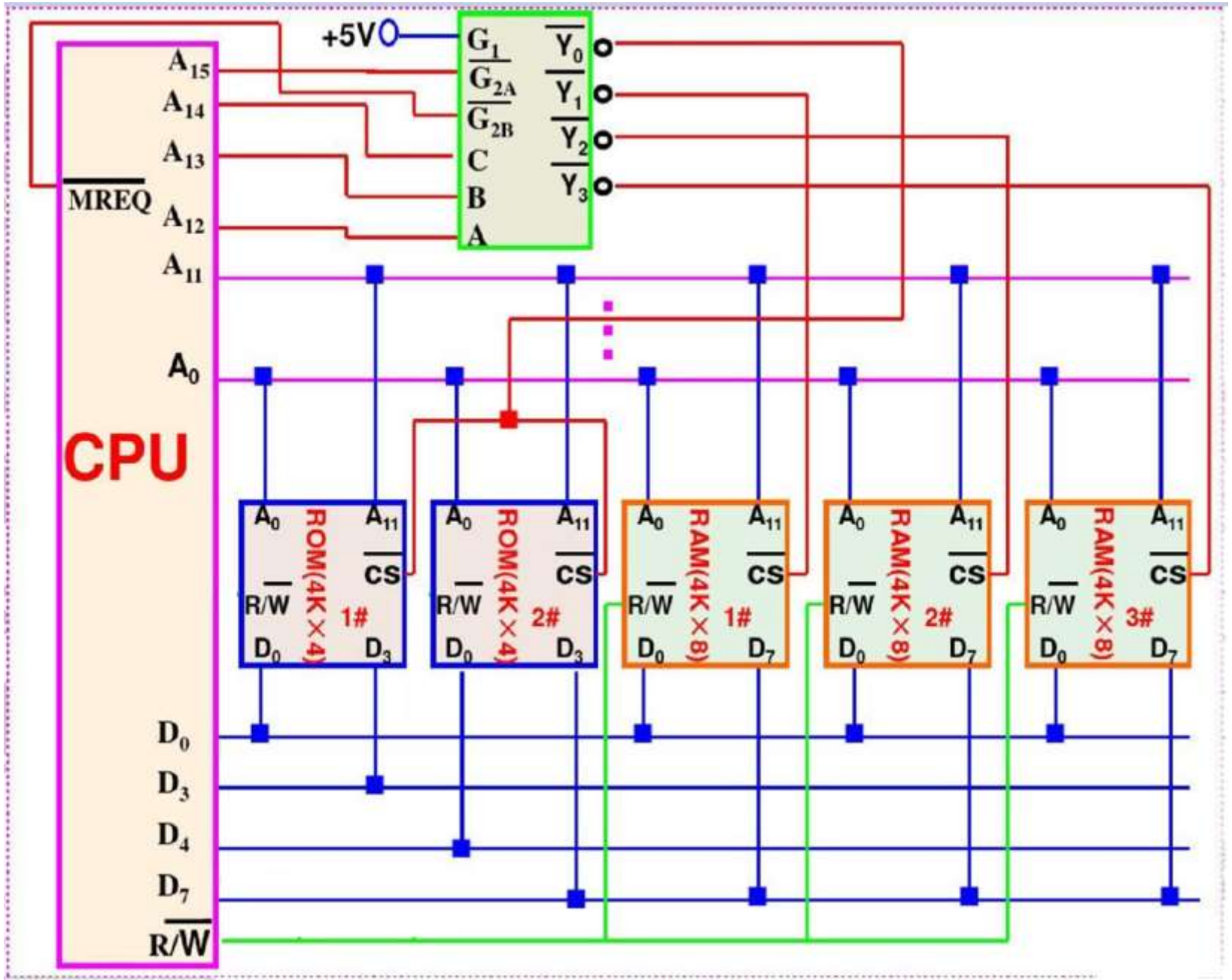
	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
ROM 1,2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
RAM 1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
RAM 2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1
RAM 3	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1

CSDN @蓝净云

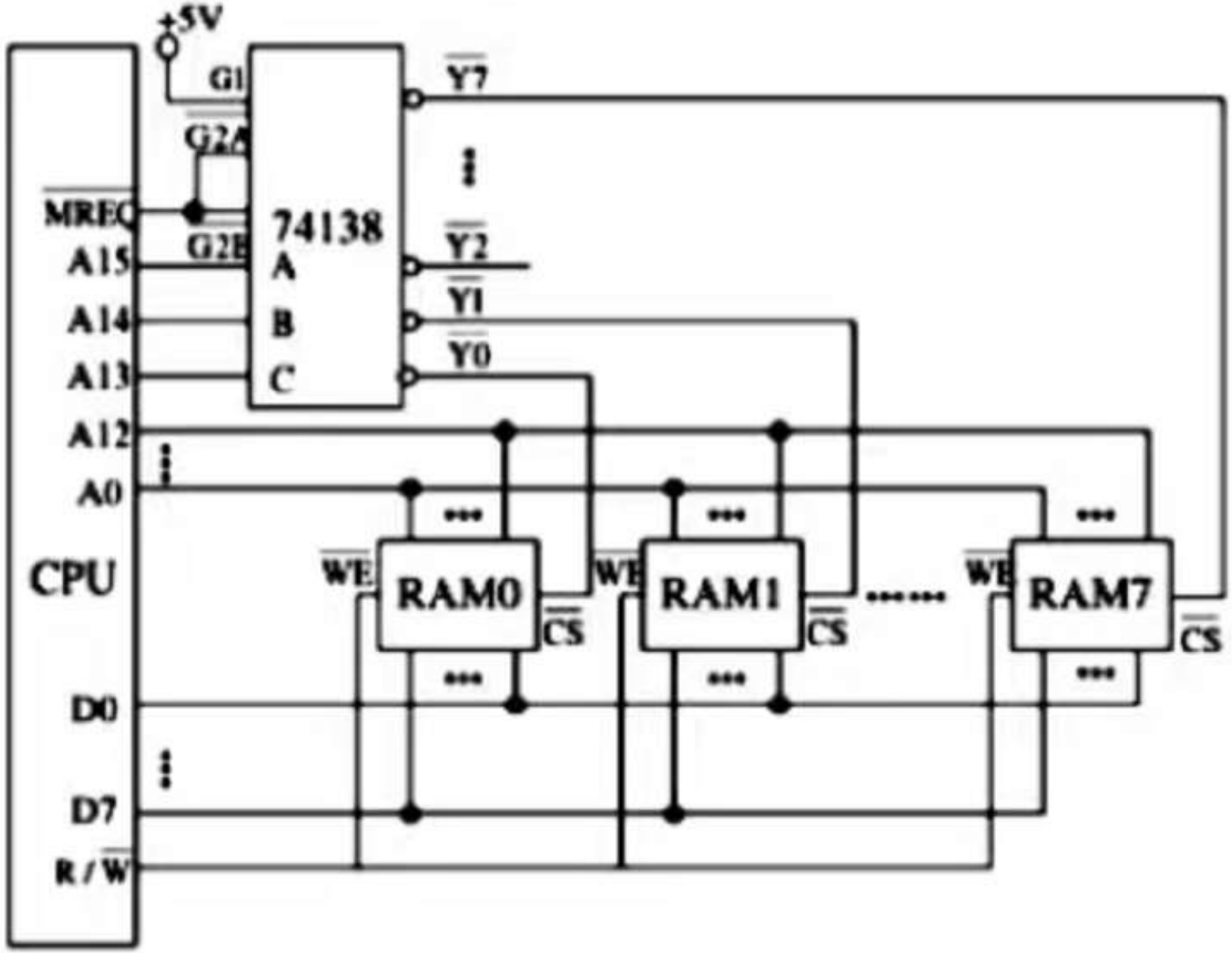
C B A

(2)选片:ROM:选择4K×4位芯片2片,位并联
RAM:选择4K×8位芯片3片,字串联
(RAM1地址范围为:1000H-1FFFH,RAM2地址范围为2000H-2FFFH, RAM3地址范围为:3000H-3FFFH)

(3)各芯片二进制地址分配如下:CPU和存储器连接逻辅图及片选逻辑如下图:



习题4.16


+5V
G1
G2A
G2B
74138
A
B
C
Y7
Y2
Y1
Y0
MREQ
A15
A14
A13
A12
A0
CPU
D0
D7
R/W
WE
CS
RAM0
RAM1
RAM7

(2)地址空间分配图:
RAM0:0000H-1FFFH
RAM1:2000H-3FFFH
RAM2:4000H-5FFFH
RAM3:6000H-7FFFH
RAM4:8000H-9FFFH
RAM5:A000H-BFFFH
RAM6:C000H-DFFFH
RAM7:E000H-FFFFH

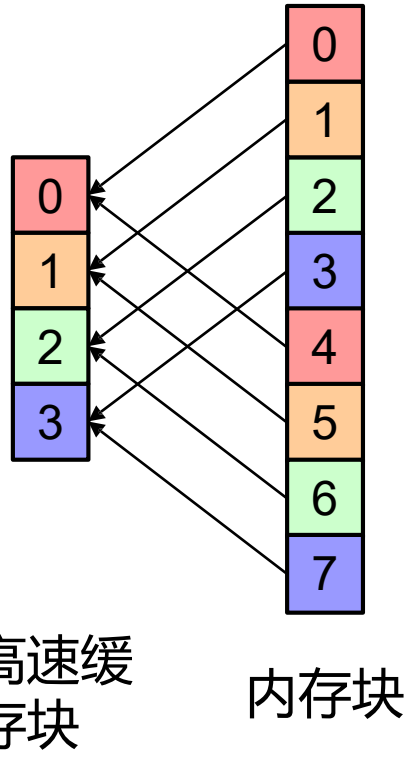
(3) 存储芯片的片选输入端很可能总是处于低电平。
假设芯片与译码器本身都是好的，可能的情况有:
①该片的$\overline{\mathrm{CS}}$ 端与$\overline{\mathrm{WE}}$ 端错连或短路:
②该片的$\overline{\mathrm{CS}}$ 端与$\overline{\mathrm{MREQ}}$端错连或短路;
③该片的$\overline{\mathrm{CS}}$ 端与地线错连或短路。

(4) A13恒为“1”的情况。此时存储器只能寻址A13=1的地址空间(奇数片)，A13=0的另一半地址空间(偶数片)将永远访问不到。若对A13=0的地址空间(偶数片)进行访问，只能错误地访问到A13=1的对应空间(奇数片)中去。

Cache-主存地址映射(10分)

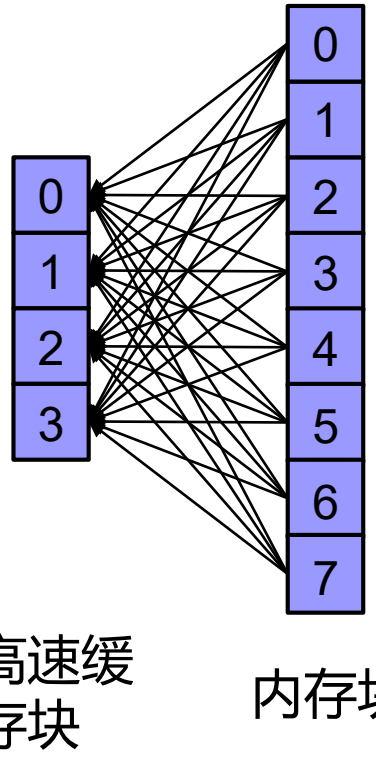
Direct

- 内存块**直接映射** (%) 到高速缓存块.



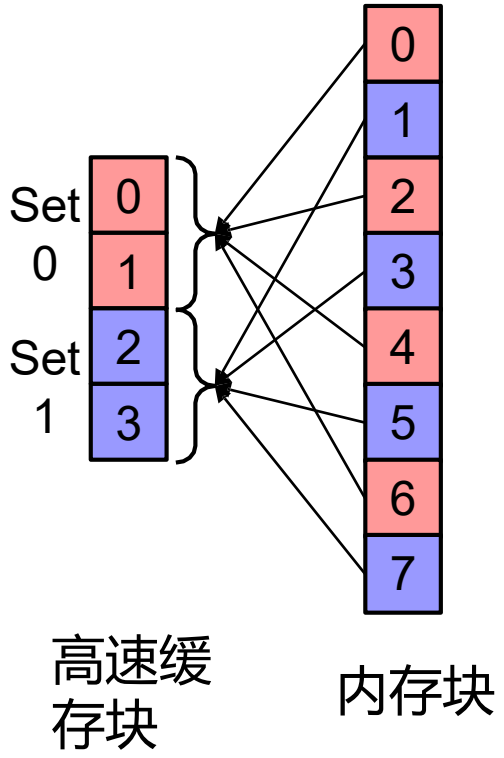
Associative

- 内存块可**映射到任何**缓存块.
  (先到先得!)



Set Associative

- 内存块**直接映射** (%) 到高速缓存**集**.
  (在一组中? Associative!)

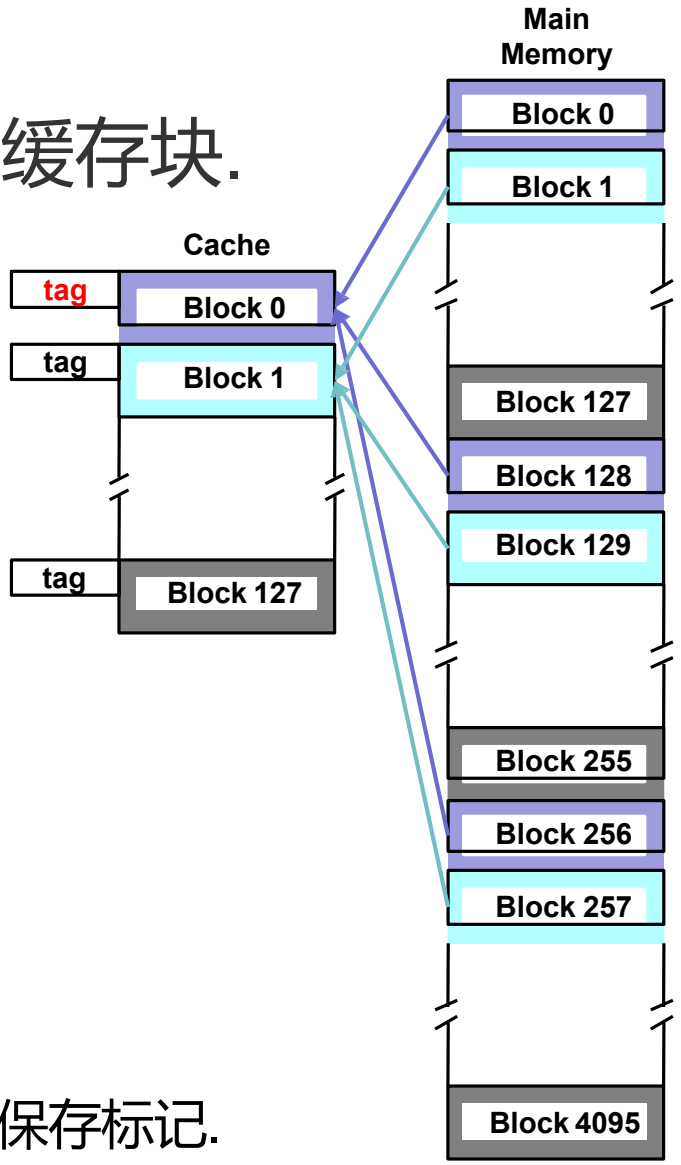


直接映射Direct Mapping

- **直接映射高速缓存**:
  每个内存块将直接映射到一个高速缓存块.
- **直接映射函数**:

MB #`j` → CB #(`j mod 128`)

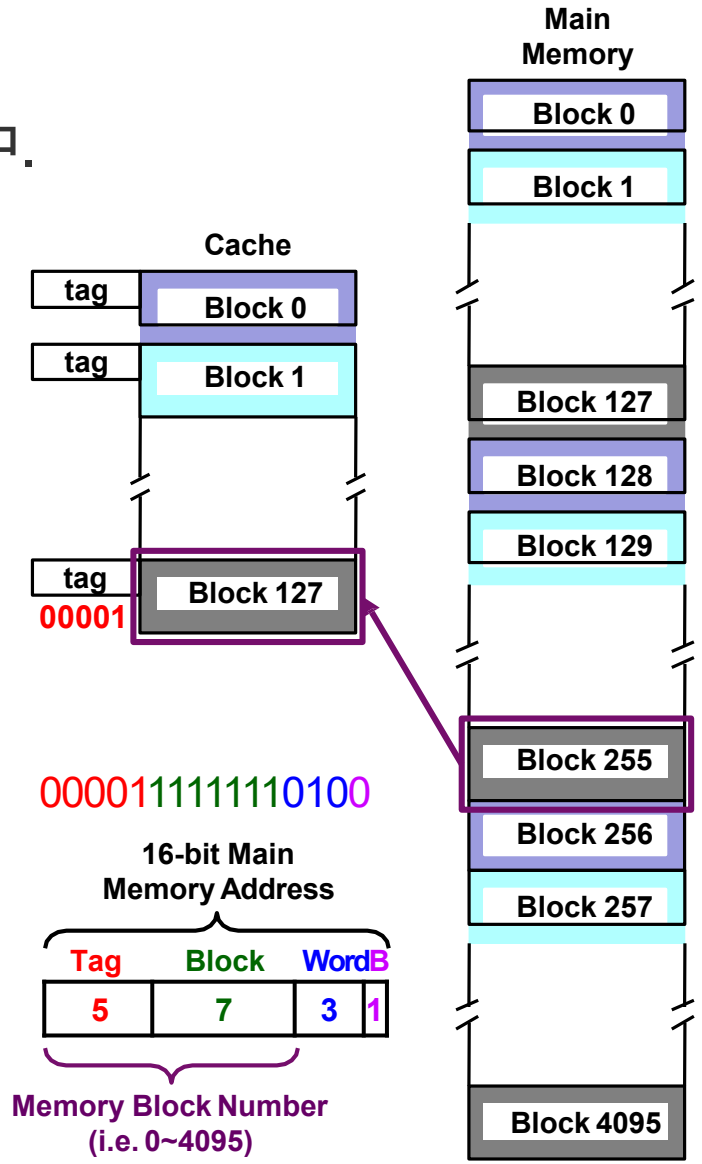
  - **128**?有 128 个高速缓存块.
  - 32 MBs 映射到 1 CB.
    - MBs **0, 128, 256, …, 3968** → CB **0**.
    - MBs **1, 129, 257, …, 3969** → CB **1**.
    - …
    - MBs **127, 255, 383, …, 4095** → CB **127**.
  - 每个 CB 都需要一个**标记**.
    - 许多 MB 将被映射到缓存中的同一个 CB.
    - 我们需要使用一些缓存空间（成本！）来保存标记.



直接映射Direct Mapping

1 Block = 2^3 Words
1 Word = 2^1 Bytes

- **技巧:** 将 16 位主存地址解释为:
  - **标记**: 跟踪哪个 MB 位于相应的 CB 中.
    - **5** 位: 16 – (**7** + **4**) = **5** 位.
  - **块**: 确定缓存中的 CB.
    - **7** 位: 有 128 = 2^7 个缓存块.
  - **字**: 选择块中的一个字.
    - **3** 位: 一个块中有 8 = 2^3 个字.
  - **字节**: 在一个字中选择一个字节.
    - **1** 位: 一个字中有 2 = 2^1 个字节.
- Ex: CPU 正在查找 $(0FF4)_{16}$
  - MAR = $(0000\ 1111\ 1111\ 0100)_2$
  - MB = $(0000\ 1111\ 1111)_2 = (255)_{10}$
  - CB = $(1111111)_2 = (127)_{10}$
  - Tag = $(00001)_2$



直接映射Direct Mapping

- 为什么前 5-位 用于 **标记**? 为什么中间 7-位 用于 **块**?

MB #j → CB #(j mod 128)

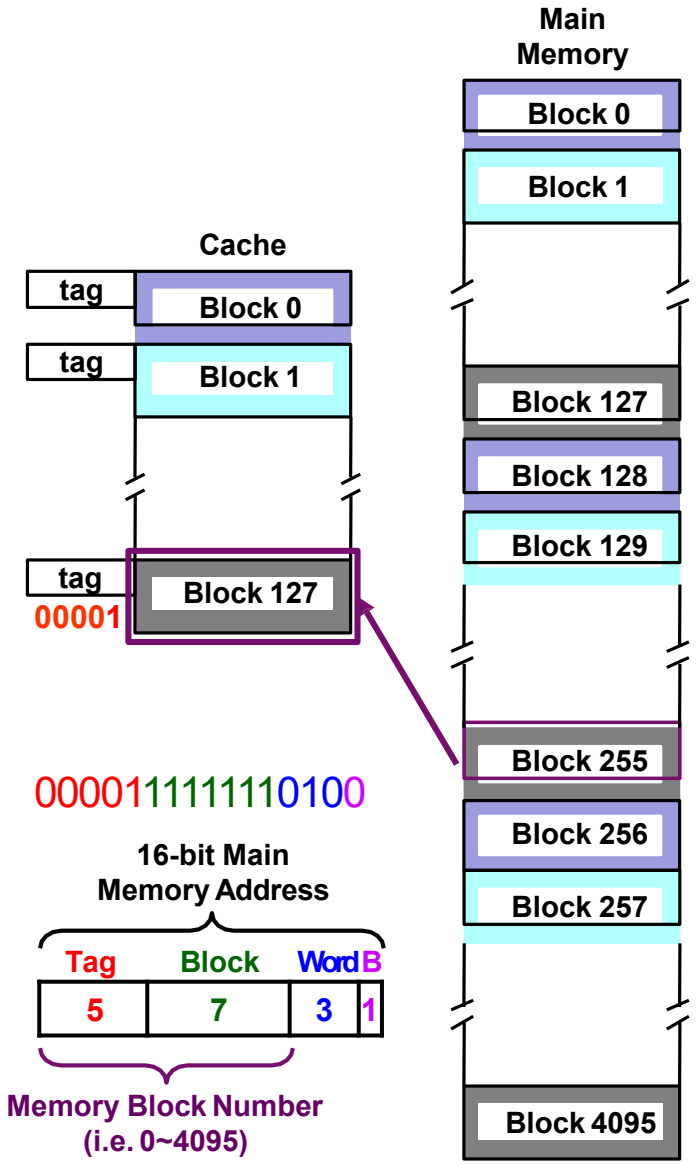
00001 Quotient

10000000) 000011111111~~0100~~

(128)$_{10}$ 10000000

1111111 Remainder

- 查找 16-位地址 (**t**, **b**, **w**, **b**):
  1. 查看 MB (**t**, **b**) 是否已在 CB **b** 中 通过将 **t** 与 CB **b** 的**标记**进行比较.
  2. 如果否, 用 MB (**t**, **b**) 替换 CB **b** 并用 **t** 更新 CB **b** 的**标记**.
  3. 最后访问 CB **b** 中的字 **w**.

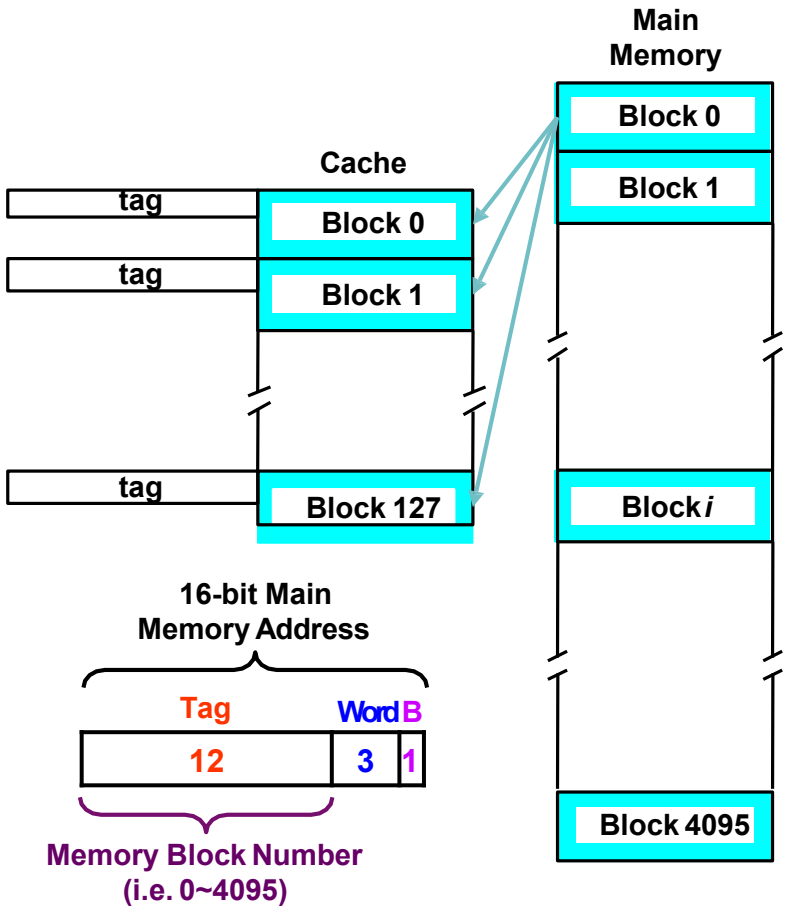


全相连映射Associative Mapping

- **Direct Mapping**: MB 受限于由 mod 操作决定的特定 CB.
- **全相连映射:**

允许 MB 映射到缓存中的任何 CB.

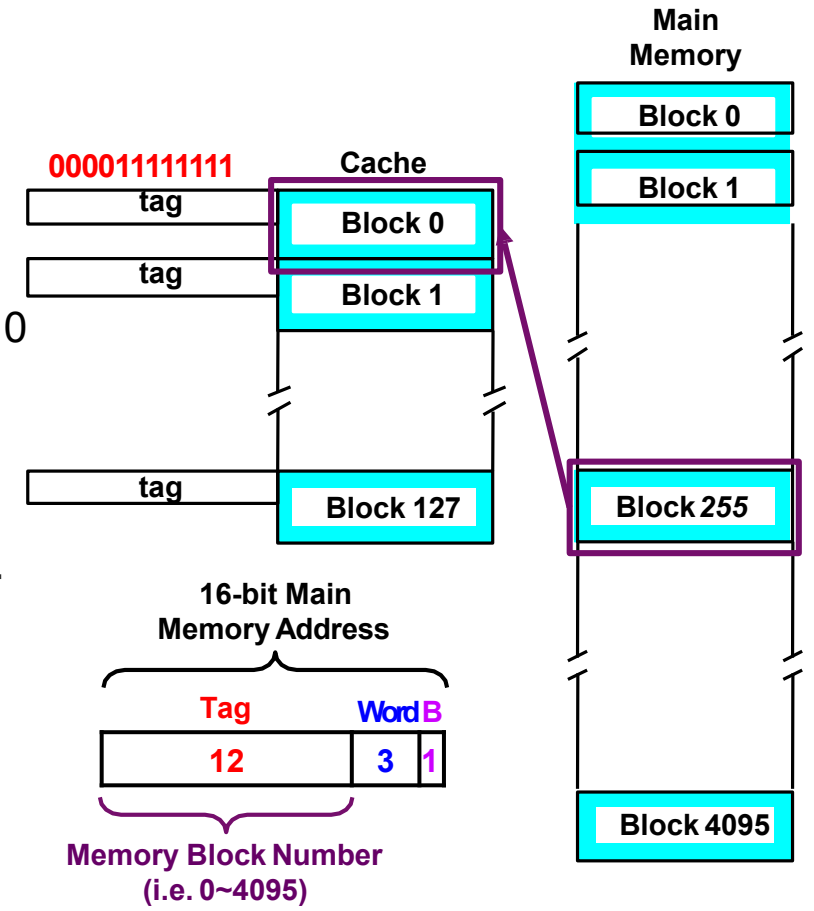
- **技巧:** 16 位主存地址解释如下:
  - **标记**: 前 **12** 位 (i.e., MB 编号) 全部用于表示 MB.
  - **字** & **字节**: 最后 **3** & **1** 位用于选择块中的字和字节.



全相连映射Associative Mapping

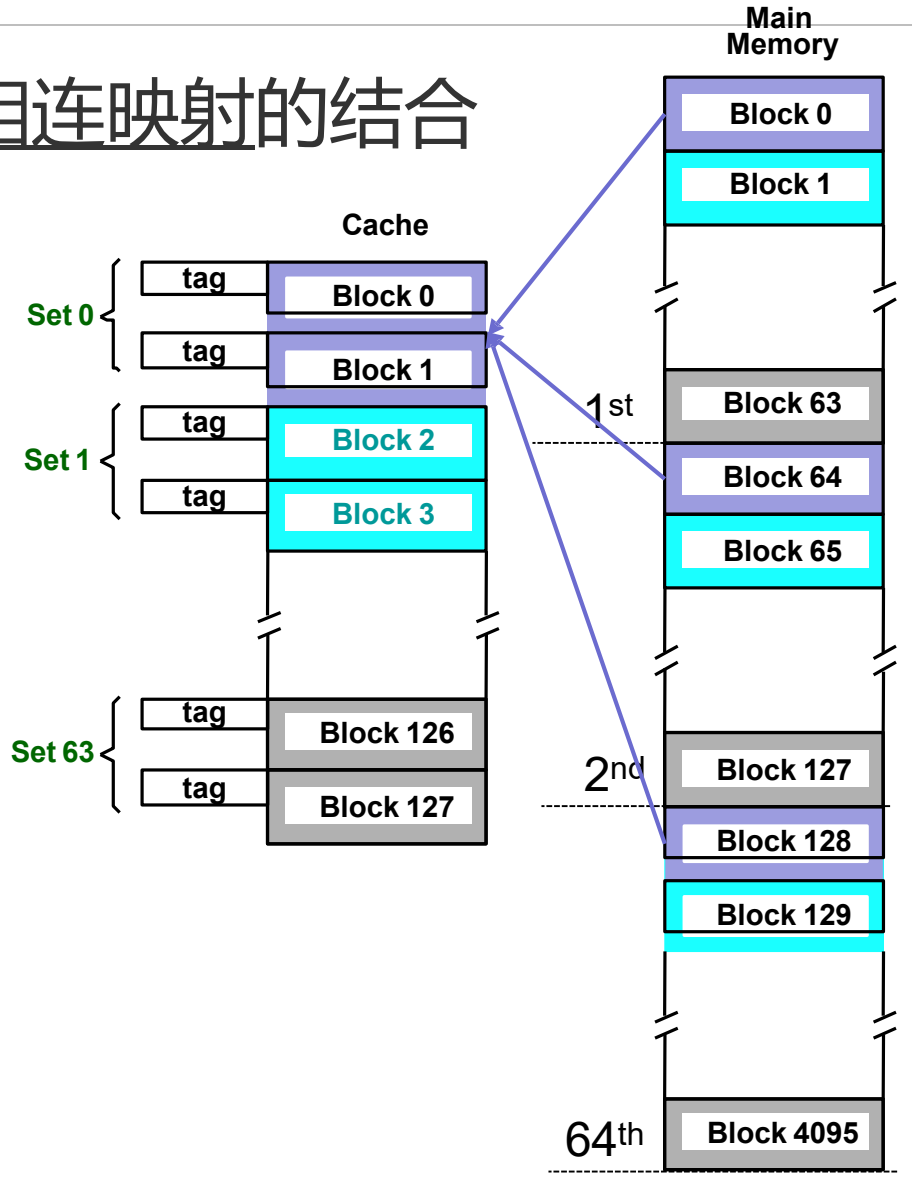
1 Block = 2^3 Words
1 Word = 2^1 Bytes

- 如何确定 CB?
  - 任何 MB 都没有预先确定的 CB.
  - 所有 CB 都以先到先得（FCFS）的方式使用.
- Ex: CPU 正在查找 $(0FF4)_{16}$
  - 假设所有 CB 都是空.
  - MAR = $(0000\ 1111\ 1111\ 0100)_2$
  - MB = $(0000\ 1111\ 1111)_2 = (255)_{10}$
    - Tag = $(0000\ 1111\ 1111)_2$
- 查找 16-位 地址. (**t**, **w**, **b**):
  - 必须将 **128 个 CB** 的**所有标记**与 **t** 进行**比较**，以确定 MB **t** 当前是否在缓存中.
    - 128 个标记的比较可以通过硬件**并行处理**（成本高）.



组相连映射Associative Mapping

- **组相连映射**: 直接映射与全相连映射的结合
  - **Direct**:首先将 MB 映射到缓存组（而不是 CB）
  - **Associative**:然后映射到高速缓存组中的任意 CB
- ***K*-way Set Associative**: 缓存组有 ***k*** 个CB.
  - Ex: 2-way set associative
    - $128 \div 2 = 64\ (sets)$
    - 对 MB #`j`, (`j mod 64`) 得出 **组** 编号.
      - E.g. MBs 0, 64, 128, …, 4032 → 缓存组 #0.



组相连映射Associative Mapping

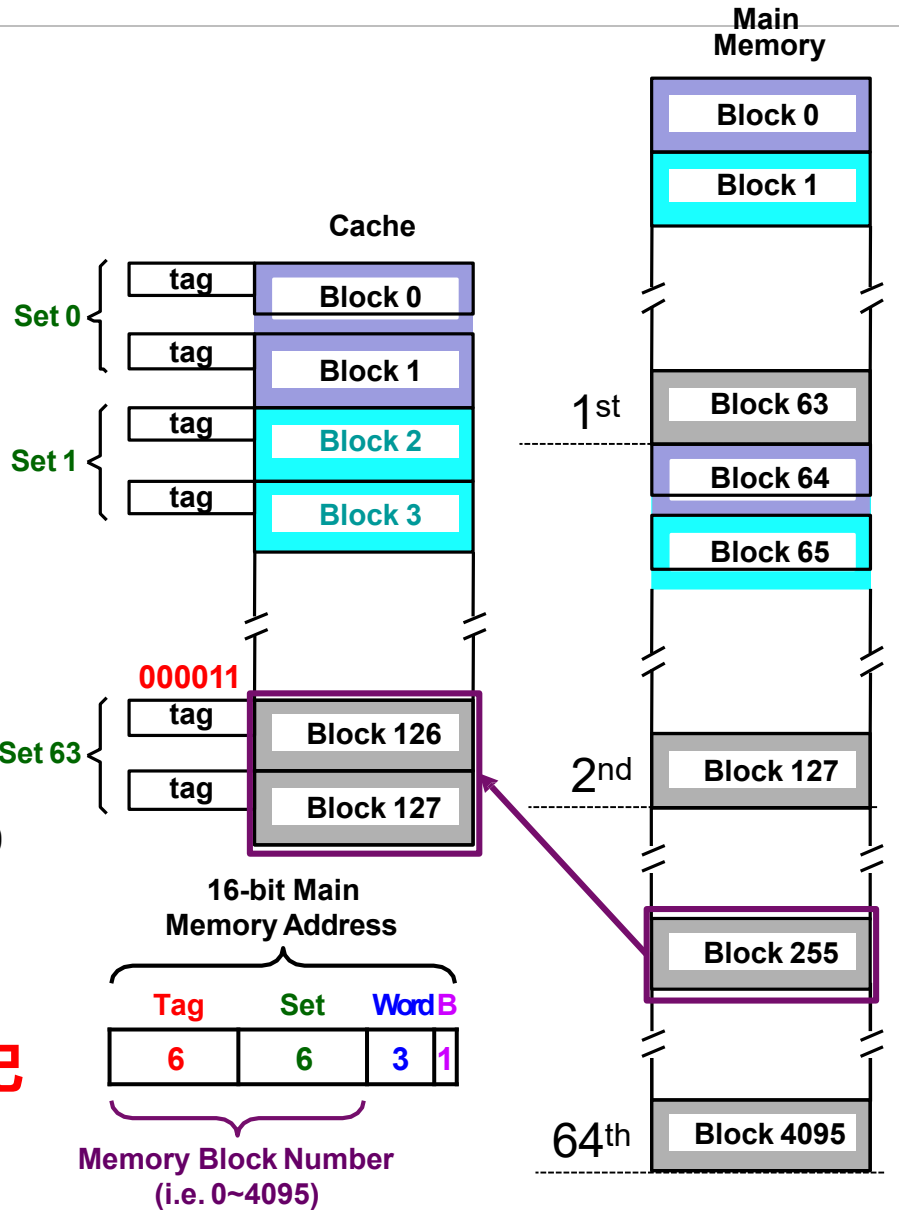
1 Block = 2^3 Words
1 Word = 2^1 Bytes

- 考虑 2-way set associative.
- **技巧**：16 位地址解释如下:
  - **标记**: 前 **6** 位 (**商**).
  - **组**: 中间 **6** 位 (**余数**).
    - **6** 位: 有 2^6 个缓存组.
  - **字** & **字节**: 最后 **3** & **1** 位.

Ex: CPU 正在查找 $(0FF4)_{16}$

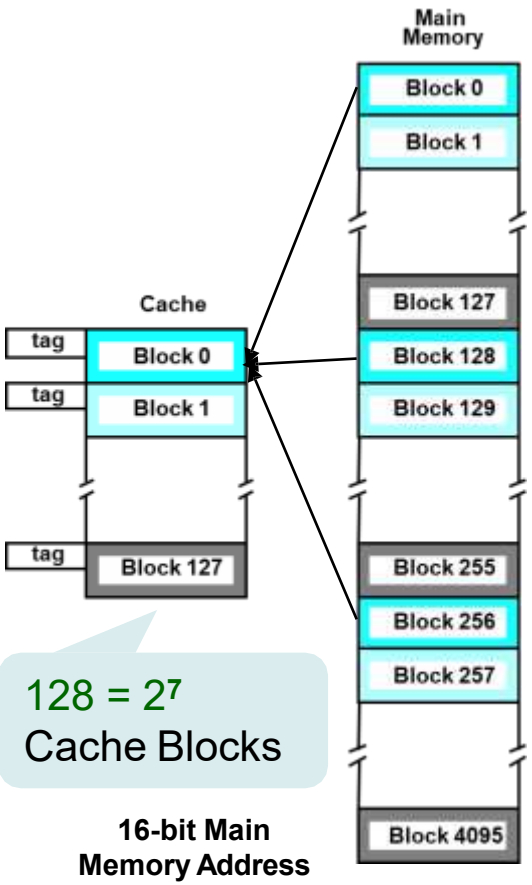
- 假设所有 CB 都是空.
- MAR = (0000 1111 1111 0100) $_2$
- MB = $(0000\ 1111\ 1111)_2 = (255)_{10}$
- Cache Set = $(111111)_2 = (63)_{10}$
- Tag = $(000011)_2$

注意：必须比较一组中 CB 的**所有标记**（由硬件并行处理）.



映射函数总结

Direct



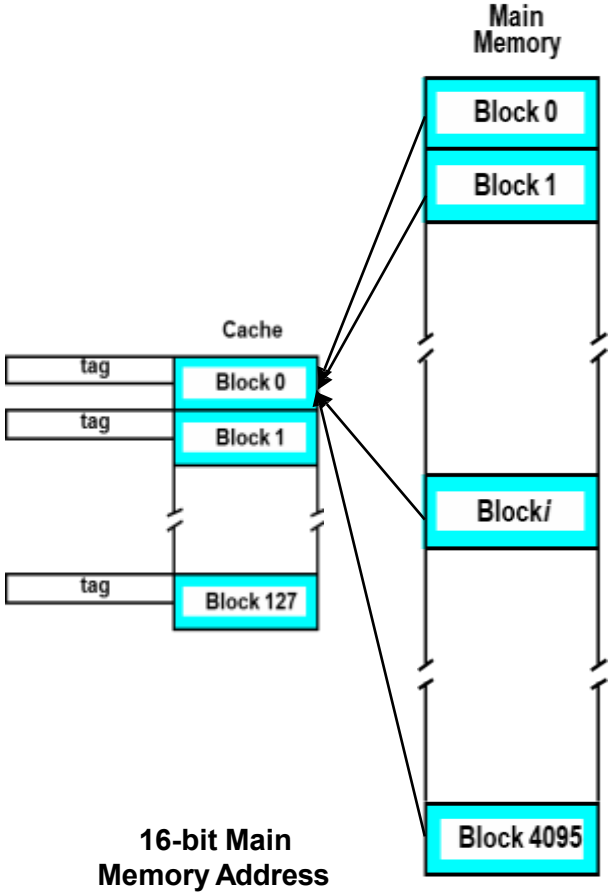
128 = 2^7
Cache Blocks

16-bit Main Memory Address

Tag	Block	Word	B
5	7	3	1

Memory Block Number
(i.e. 0~4095)

Associative



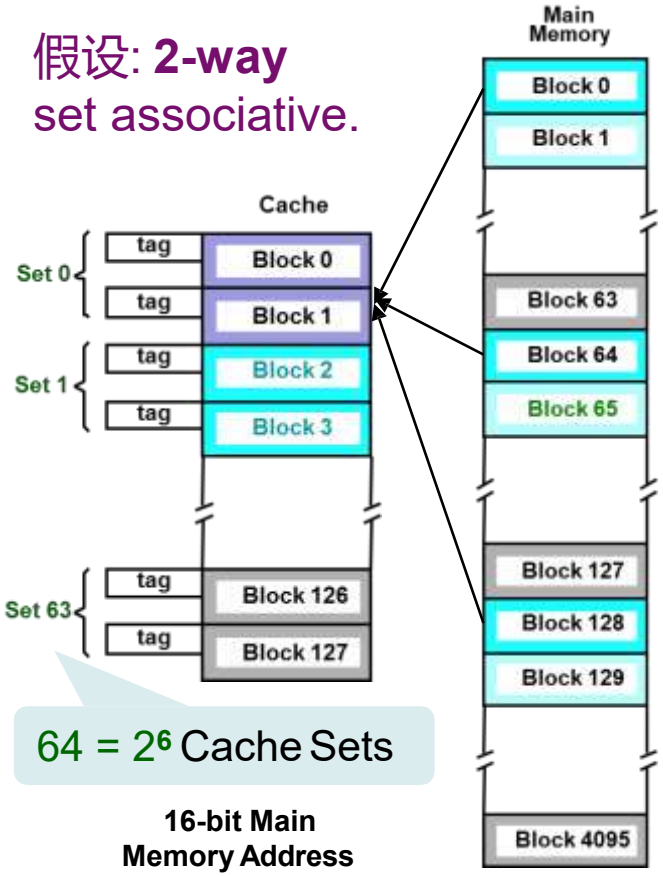
16-bit Main Memory Address

Tag	Word	B
12	3	1

Memory Block Number
(i.e. 0~4095)

Set Associative

假设: **2-way** set associative.



64 = 2^6 Cache Sets

16-bit Main Memory Address

Tag	Set	Word	B
6	6	3	1

Memory Block Number
(i.e. 0~4095)

1 Block = 2^3 Words
1 Word = 2^1 Bytes

例4.8 假设主存容量 512 KB，Cache 容量 4 KB，每个字块 16 个字，每个字 32 位。

（1）Cache 地址多少位？可容纳多少块？

4 KB：12位

4 KB / 4 B = 1 K　　1 K / 16 = 64

（2）主存地址多少位？可容纳多少块？

512 KB：19位

512 KB / 4 B = 128 K

128 K / 16 = 8192

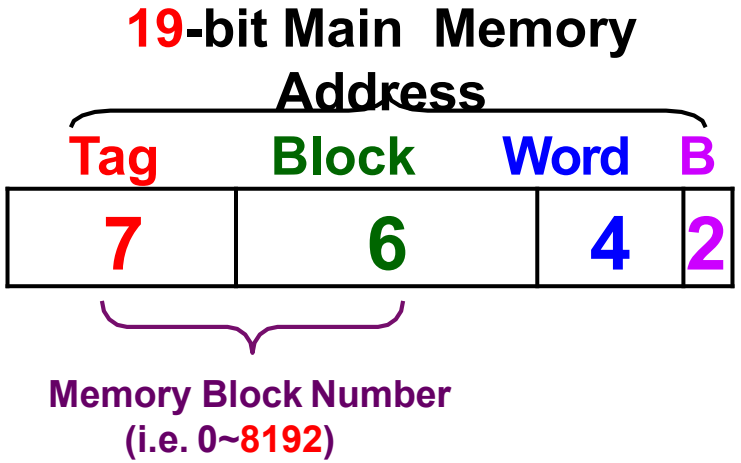
（3）在直接映射方式下，主存第几块映射到Cache 的第5块（从1开始）？

MB #j → CB #(j mod 64)

j % 64 = 5

j 的取值：5、64+5、64*2+5… 64*127+5

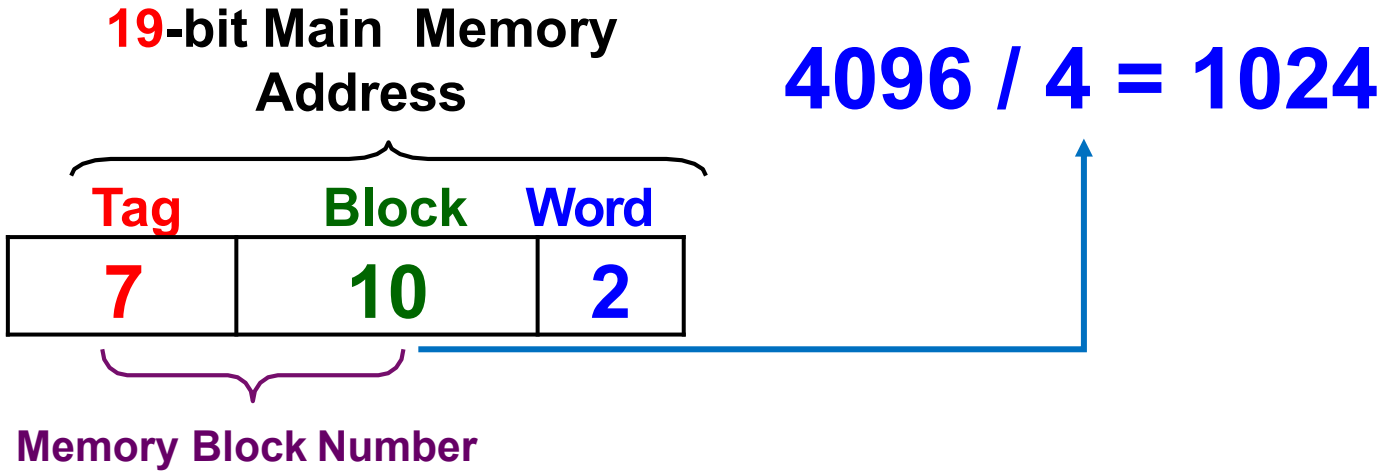
（4）在直接映射方式下，主存地址中各段的位数？



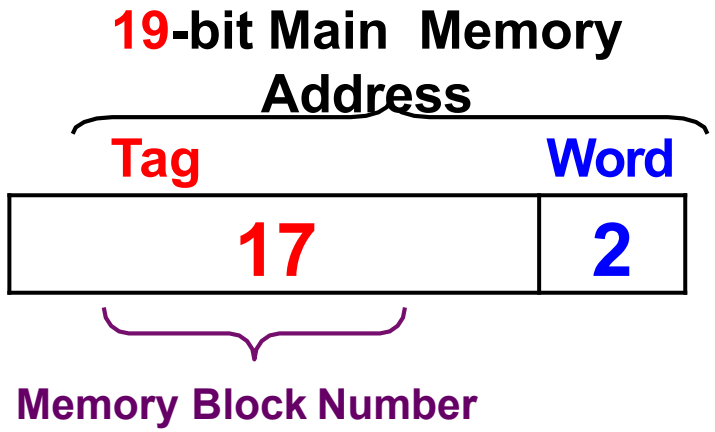
例4.9 假设主存容量 512 K×16位，Cache 容量 4096 × 16位，每个块 4 个 16位的字，访存地址为字地址。

（1）直接映射方式，主存地址格式

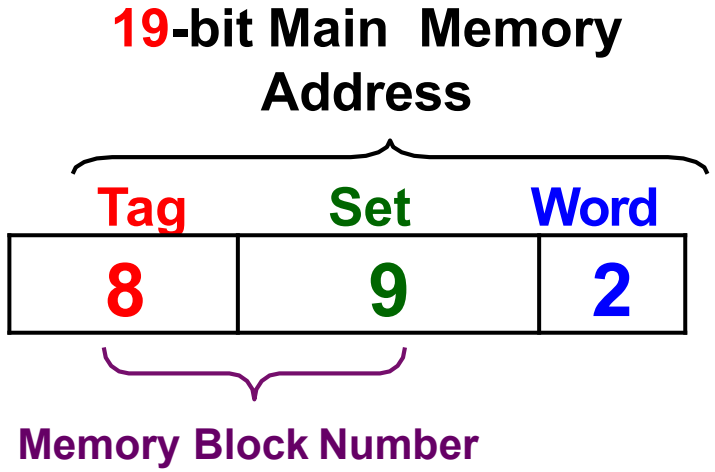
512 K × 16位：主存地址 19 位



（2）全相连映射方式，主存地址格式

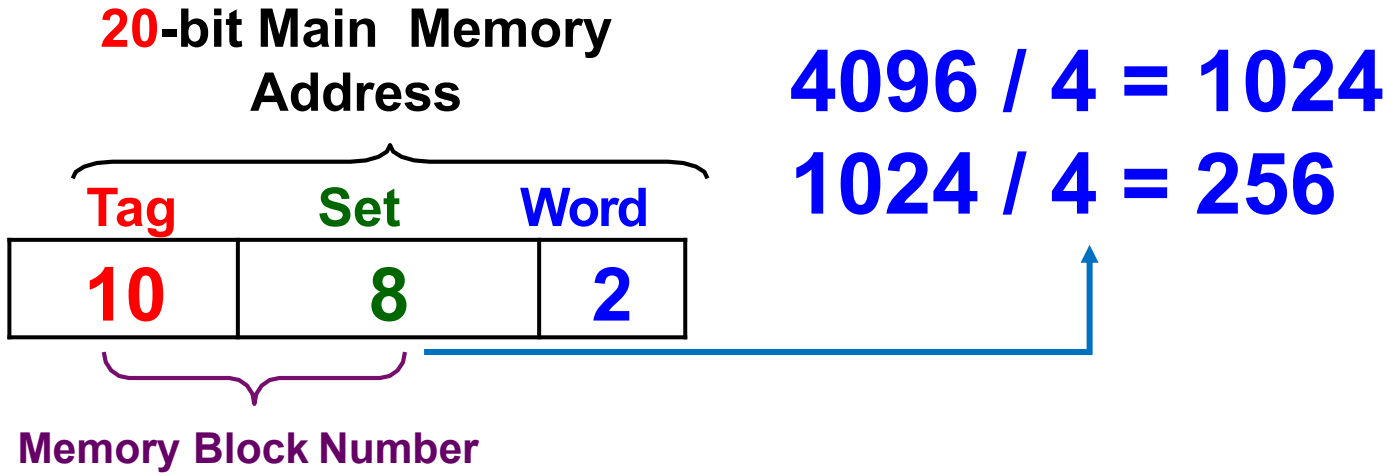


（3）二路组相连映射方式，主存地址格式



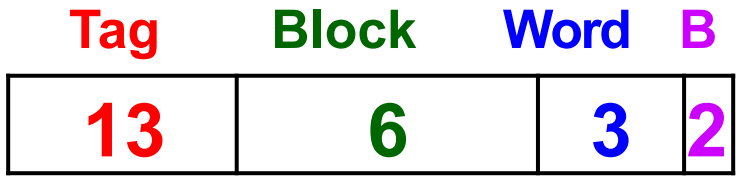
（4）主存容量 512 K×32位，块长不变，四路组相连映射方式，主存地址格式

1024 K × 16位：主存地址 20 位



例4.11

（1）四路组相连，主存地址格式



（2）cache和主存之间的传输是一块一块的

第 1 次：0、8、16、24、。。。。、96，共 13 次未命中

第 2 次~第 10 次全命中

$$\frac{100*10-13}{100*10}=98.7\%$$

（3）设cache存储周期为 t，则主存存储周期为 5t

$$\frac{1000*5t}{(1000-13)*t+13*5t}-1=3.75$$

（4）效率

$$e=\frac{t_c}{t_a}=\frac{t}{0.987*t+(1-0.987)*5t}=95\%$$

t_c为命中时cache的访问时间，t_a为平均访问时间

输入输出系统

10分

例题5.2、书后习题5.2、5.3、5.4

CPU响应中断的条件和时间

单重中断、多重中断（图5.43）

与第8章 8.4节中断一起看

屏蔽字技术（例8.2）

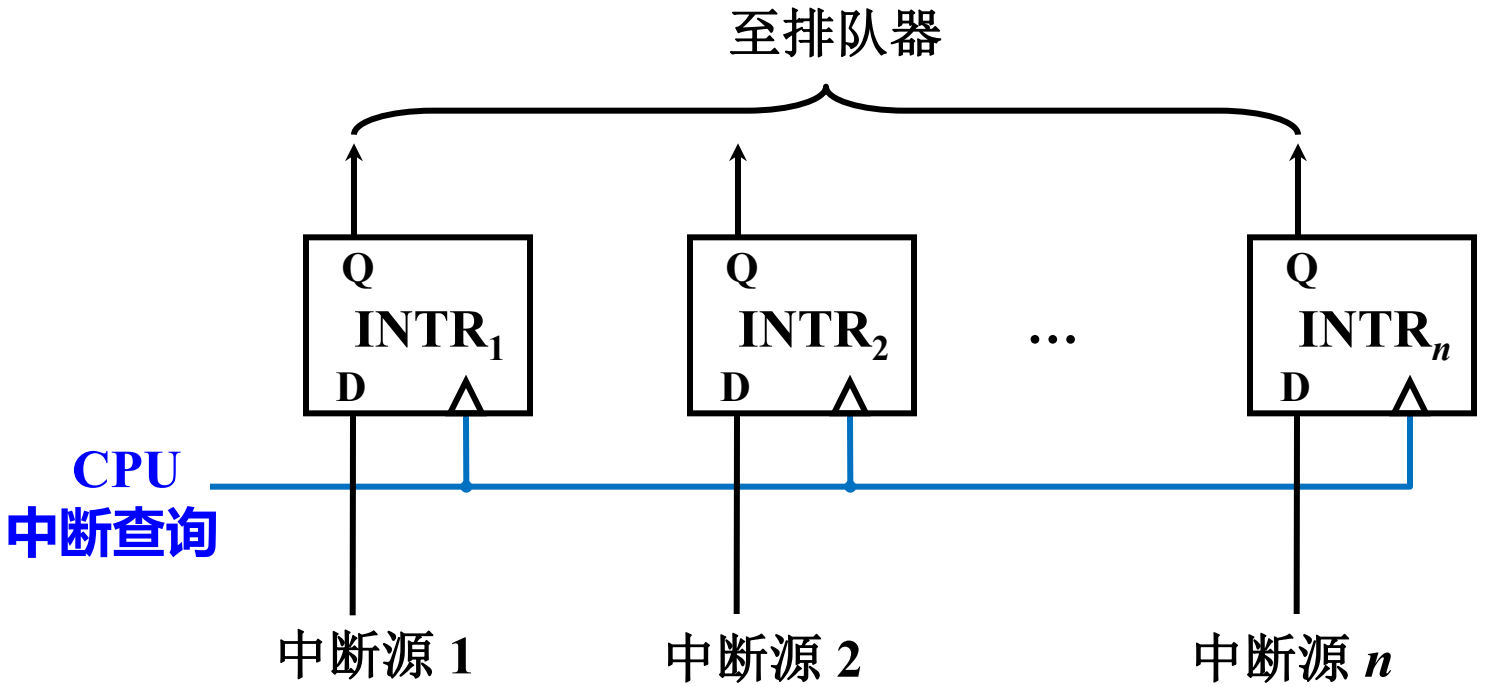
中断响应

1. 响应中断的 **条件**

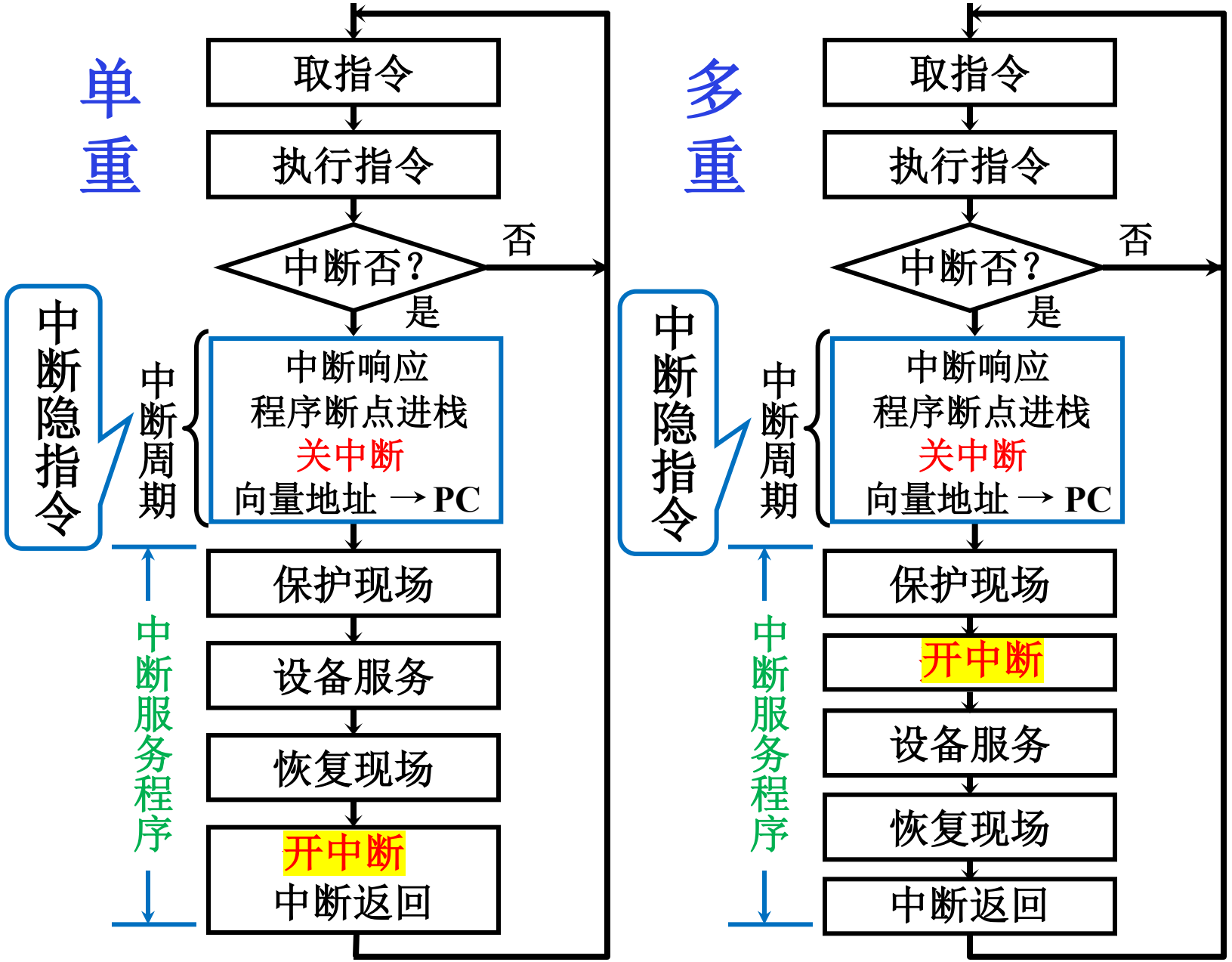
   允许中断触发器　**EINT = 1**

2. 响应中断的 **时间**

   指令执行周期结束时刻由CPU 发查询信号



单重中断和多重中断的服务程序流程



屏蔽技术可改变处理优先等级

响应优先级　　不可改变

处理优先级　　可改变（通过重新设置屏蔽字）

中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

响应优先级 **A→B→C→D** 降序排列

处理优先级 **A→D→C→B** 降序排列

指令系统

指令格式设计(10分)

- **例.** 字长16位，主存64K，指令单字长单地址，80条指令。寻址方式有直接、间接、相对、变址。请设计指令格式并计算每种寻址方式能访问的主存空间范围。

- 80条指令 ⇒ OP字段需7位（$2^7=128$）
- 4种寻址方式 ⇒ 寻址方式位需2位
- 单字长单地址 ⇒ 地址码长度=16-7-2 =7位



- 直接寻址：0~127
- 间接寻址：0~65535
- 相对寻址：0~65535（PC为16位）
- 变址寻址：0~65535（X为16位）

例7.5

习题7.16

（1）一地址指令格式



（2）直接寻址范围：2^6 (0~63)

（3）一次间接寻址范围：2^{16} (0~65535)
多次间接寻址范围：2^{15} (0~32767)

（4）立即数范围：-32～31（有符号数），
或0～63（无符号数）

（5）相对寻址位移量：-32～31

（6）执行时间最短：立即寻址

执行时间最长：间址寻址

- 需多次访存（一次间接寻址要两次访存，多次间接寻址要多次访存）

便于程序浮动：相对寻址

- 操作数位置可随程序存储区的变动而改变，操作数的有效地址只与当前指令地址相差一定的位移量

适合数组处理：变址寻址

- 变址寄存器的内容由用户给定，而且在程序的执行过程中允许用户修改，而其形式地址始终不变，（变址值可自动修改而不需要修改程序）

（7）如何使寻址范围扩大到 4M

需要有效地址22位，此时可将单字长一地址的格式改为双字长



（8）也需要有效地址 22 位，比如可以 16 位的基址寄存器左移 6 位再和形式地址A相加

CPU的结构和功能

流水线性能（10分）：例8.1、书后习题8.12

1. 吞吐率

单位时间内 **流水线所完成指令** 或 **输出结果** 的 **数量**

设 m 段的流水线各段时间为Δt

- 最大吞吐率

$$T_{pmax} = \frac{1}{\Delta t}$$

- 实际吞吐率

连续处理 n 条指令的吞吐率为

$$T_p = \frac{n}{m \cdot \Delta t + (n-1) \cdot \Delta t}$$

2. 加速比 S_p

m 段的 **流水线的速度** 与等功能的 **非流水线的速度** 之比

设流水线各段时间为 Δt

完成 n 条指令在 m 段流水线上共需

$$T = m \cdot \Delta t + (n\text{-}1) \cdot \Delta t$$

完成 n 条指令在等效的非流水线上共需

$$T' = nm \cdot \Delta t$$

则

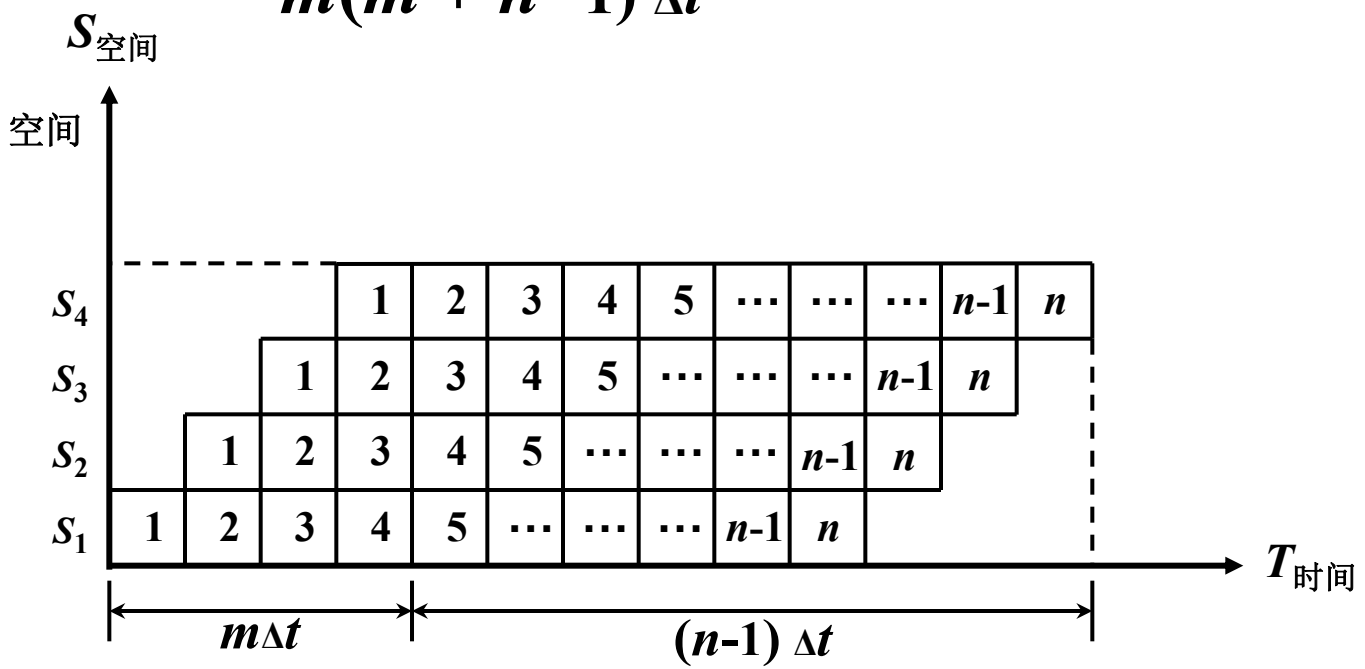
$$S_p = \frac{nm \cdot \Delta t}{m \cdot \Delta t + (n\text{-}1) \cdot \Delta t} = \frac{nm}{m + n - 1}$$

3. 效率

流水线中各功能段的 **利用率**

$$\text{效率} = \frac{\text{流水线各段处于工作时间的时空区}}{\text{流水线中各段总的时空区}}$$

$$= \frac{mn\Delta t}{m(m+n-1)\,\Delta t}$$



例8.1 假设指令流水线分 IF、ID、EX、WR 4个阶段，共有10条指令连续输入流水线。

（1）画指令周期流程

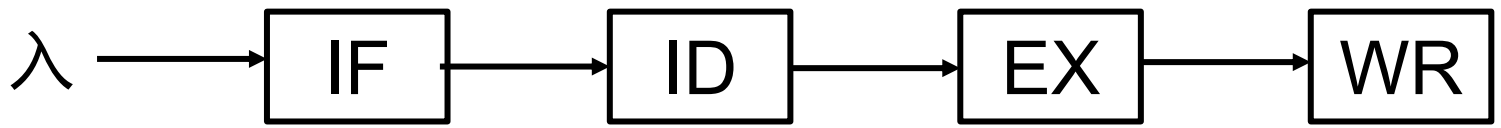
（2）画非流水线时空图

（3）画流水线时空图

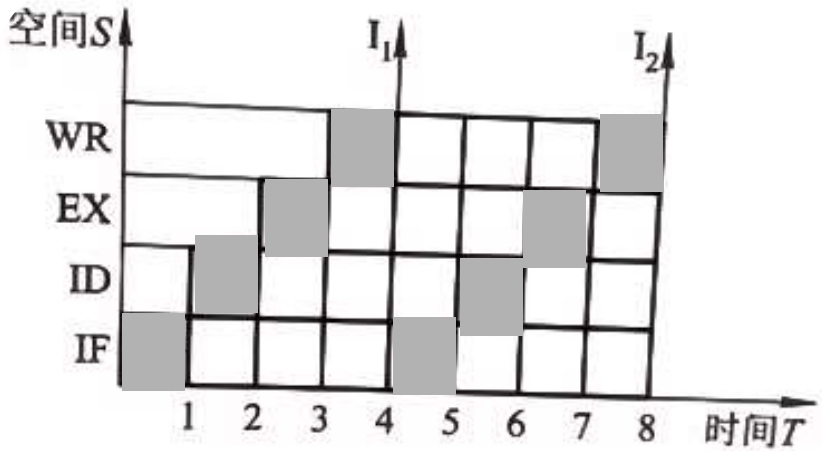
（4）假设时钟周期为 100 ns，求流水线实际吞吐率

（5）求加速比

（1）画指令周期流程

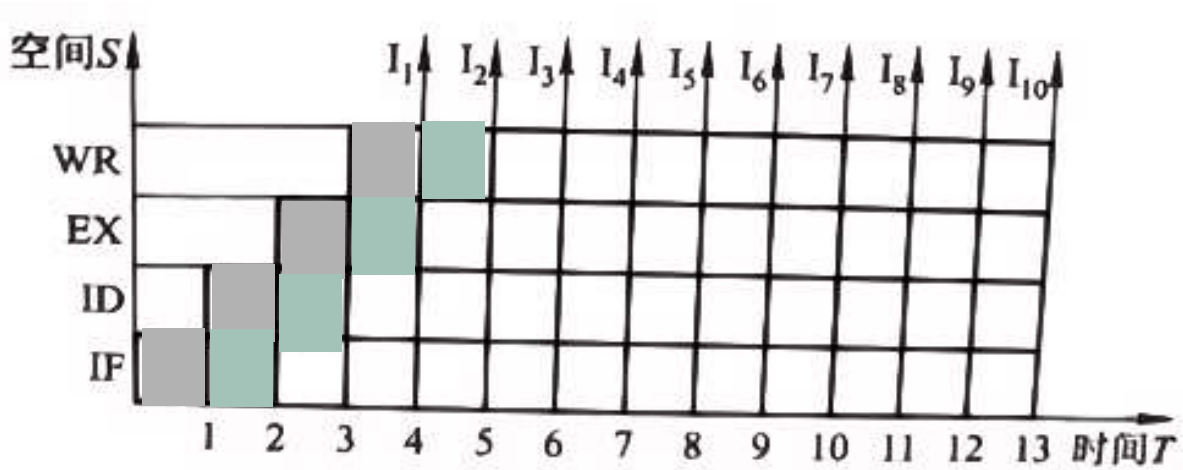


(2) 画非流水线时空图



(b) 非流水线时空图

(3) 画流水线时空图



(c) 标准流水线时空图

（4）假设时钟周期为 100 ns，求流水线实际吞吐率

$$T_p = \frac{n}{m \cdot \Delta t + (n-1) \cdot \Delta t}$$

$$= \frac{10}{4 \times 100 \times 10^{-9} + (10-1) \times 100 \times 10^{-9}}$$

$$= 0.77 \times 10^7$$

（5）求加速比

$$S_p = \frac{nm \cdot \Delta t}{m \cdot \Delta t + (n-1) \cdot \Delta t} = \frac{nm}{m + n - 1}$$

$$= \frac{10 \times 4}{4 + 10 - 1} = 3.08$$

习题8.12　流水线时钟周期至少为10 ns

（1）求加速比

$$S_p = \frac{nm \cdot \Delta t}{m \cdot \Delta t + (n\text{-}1) \cdot \Delta t}$$

$$= \frac{(10+8+10+10+7)*12}{5*10+(12-1)*10}$$

$$= 3.375$$

（2）求实际吞吐率

$$T_p = \frac{n}{m \cdot \Delta t + (n\text{-}1) \cdot \Delta t}$$

$$= \frac{12}{5*10+(12-1)*10}$$

$$= 0.075 \quad \text{条} / ns$$

控制单元的功能

- 例9.1、例9.2（10分）

计算机的运算方法

- 浮点数加减（15分）、浮点数乘法（10分）

规格化

(1) 规格化数的定义

$$r = 2 \qquad \frac{1}{2} \leq |S| < 1$$

(2) 规格化数的判断

$S>0$	规格化形式	$S<0$	规格化形式
真值	0.1××…×	真值	−0.1××…×
原码	0.1××…×	原码	1.1××…×
补码	0.1××…×	补码	1.0××…×
反码	0.1××…×	反码	1.0××…×

原码　不论正数、负数，第一数位为1

补码　符号位和第一数位不同

规格化特例

$$S = -\frac{1}{2} = -0.100\cdots0$$

$$[S]_{原} = 1.100\cdots0$$

$$[S]_{补} = \boxed{1.1}00\cdots0$$

$\therefore$ $[-\frac{1}{2}]_{补}$ 不是规格化的数

$$S = -1$$

$$[S]_{补} = \boxed{1.0}00\cdots0$$

$\therefore$ $[-1]_{补}$ 是规格化的数

6.26 (2) $x = -0.100010 \times 2^{-011}$　　$y = (-0.011111) \times 2^{-010}$

求 $x+y$（阶符、数符取2位，阶码取 3 位，尾数取 6 位）

解：　$[x]_{补} = 11, 101; 11.011110$

$[y]_{补} = 11, 110; 11.100001$

① 对阶

$$[\Delta j]_{补} = [j_x]_{补} - [j_y]_{补} \quad = \quad 11, 101$$
$$+ \quad 00, 010$$
$$11, 111$$

阶差为 –1　　$\therefore S_x \longrightarrow 1, \quad j_x + 1$

$\therefore$ **$[x]_{补}' = 11, 110; 11.101111$**

② 尾数求和

$$
\begin{array}{rl}
 & [S_x]_{补'} = 11.101111 \\
+ & [S_y]_{补} = 11.100001 \\
\hline
 & 11.010000
\end{array}
$$

$$[x+y]_{补} = 11,110;\ 11.010000$$

③ 规格化

已是规格化数

④ 0 舍 1 入

$$[x+y]_{补} = 11,110;\ 11.010000$$ **无溢出**

6.26 (3) $x = (-0.100101) \times 2^{101}$ $y = (-0.001111) \times 2^{100}$

求 $x-y$（阶符、数符取2位，阶码取 3 位，尾数取 6 位）

解：$[x]_{补} = 00, 101; 11.011011$

$[y]_{补} = 00, 100; 11.110001$

① 对阶

$$[\Delta j]_{补} = [j_x]_{补} - [j_y]_{补} = 00, 101$$
$$+ \ 11, 100$$
$$00, 001$$

阶差为 1　　$\therefore S_y \longrightarrow 1, \quad j_y + 1$

$\therefore [y]_{补'} = 00, 101; 11.1110001$

② 尾数求和

$$[S_x]_{补} = 11.011011$$
$$+[-S_y]_{补'} = 00.0001111$$
$$\overline{\qquad\qquad\qquad\qquad} $$
$$11.1000101$$

$$[x-y]_{补} = 00,101;\ 11.1000101$$

③ 规格化（左规，左移1位）

$$[x-y]_{补} = 00,100;\ 11.0001010$$

④ 0 舍 1 入

$$[x-y]_{补} = 00,100;\ 11.000101$$ 无溢出

例 6.30 $x = (-0.101000) \times 2^{-101}$ $y = (0.111011) \times 2^{-100}$

求 $x-y$（阶符、数符取2位，阶码取 3 位，尾数取 6 位）

解：$[x]_{补} = 11,011;\ 11.011000$

$[y]_{补} = 11,100;\ 00.111011$

① 对阶

$$[\Delta j]_{补} = [j_x]_{补} - [j_y]_{补} = 11,011 + 00,100 = 11,111$$

阶差为 –1　　$\therefore S_x \longrightarrow 1,\ j_x+1$

$\therefore [x]_{补}' = 11,100;\ 11.101100$

② 尾数求和

$$[S_x]_{补'} = 11.101100$$
$$+\ [-S_y]_{补} = 11.000101$$
$$\overline{\qquad\qquad\qquad}$$
$$110.110001$$

③ 规格化（右规）

$$[x-y]_{补} = 11,100;\ 10.110001$$

右规后

$$[x-y]_{补} = 11,101;\ 11.011000\ 1$$

④ 0 舍 1 入

$$[x-y]_{补} = 11,101;\ 11.011001$$

无溢出

6.29（1）$x = 0.101101 \times 2^{-100}$　　$y = (-0.110101) \times 2^{-011}$

求 $x \cdot y$（阶码取 3 位，尾数取 6 位，阶码用移码、尾数用补码）

解：　$[x]_{阶移尾补} = 00, 100; 00.101101$　　$[-x]_{尾补} = 11.010011$

$[y]_{阶移尾补} = 00, 101; 11.001011$　　$[y]_{阶补} = 11, 101$

① 阶码运算

$$\begin{array}{r} 00, 100 \\ +\ 11, 101 \\ \hline 00, 001 \end{array}$$

② 尾数相乘

A	Q(乘数)	Q_{-1}	M(被乘数)	
0000000	1001011	0	0101101	10 -> sub (A-M)
1010011	1001011	0	0101101	shift
1101001	1100101	1	0101101	11 -> nop, shift
1110100	1110010	1	0101101	01 -> add (A+M)
0100001	1110010	1	0101101	shift
0010000	1111001	0	0101101	10 -> sub (A-M)
1100011	1111001	0	0101101	shift
1110001	1111100	1	0101101	01 -> add (A+M)
0011110	1111100	1	0101101	shift
0001111	0111110	0	0101101	00 -> nop, shift
0000111	1011111	0	0101101	10 -> sub (A-M)
1011010	1011111	0	0101101	shift
1101101	0101111	1	0101101	done



$$[x \cdot y]_{\text{阶移尾补}} = 00,\ 001;\ 11.011010101111$$

③ 规格化（左规）

已是规格化

④ 舍入

$$[x \cdot y]_{\text{阶移尾补}} = 00,\ 001;\ 11.\ 011011$$

$$x \cdot y = (-0.100101) \times 2^{-111}$$

乘法舍入

- （1）无条件丢掉，称为截断处理，处理简单，但影响精度。
- （2）舍入处理
- **① 原码**
  - 0 舍 1 入法
- **② 补码**
  - Ⅰ 丢失的各位数中最高位为 0 ， 丢掉
  - Ⅱ 丢失的各位数中最高位为 1，且以下各位全为 0，丢掉
  - Ⅲ 丢失的各位数中最高位为 1，且以下各位不全为 0，在尾数最末位加 1 修正。

6.29（2）$x = -0.100111 \times 2^{-011}$　　$y = (-0.101011) \times 2^{101}$

求 $x \cdot y$（阶码取 3 位，尾数取 6 位，阶码用移码、尾数用补码）

解：　$[x]_{阶移尾补} = 00, 101; 11.011001$　　$[-x]_{尾补} = 00.100111$

$[y]_{阶移尾补} = 11, 101; 11.010101$　　$[y]_{阶补} = 00, 101$

① 阶码运算

$$\begin{array}{r} 00, 101 \\ +\ 00, 101 \\ \hline 01, 010 \end{array}$$

无溢出

② 尾数相乘

A	Q(乘数)	Q_{-1}	M(被乘数)	$[-x]_{尾补} = 00.100111$
0000000	1010101	0	1011001	10 -> sub (A-M)
0100111	1010101	0		shift
0010011	1101010	1		01 -> add (A+M)
1101100	1101010	1		shift
1110110	0110101	0		10 -> sub (A-M)
0011101	0110101	0		shift
0001110	1011010	1		01 -> add (A+M)
1100111	1011010	1		shift
1110011	1101101	0		10 -> sub (A-M)
0011010	1101101	0		shift
0001101	0110110	1		01 -> add (A+M)
1100110	0110110	1		shift

A	Q(乘数)	Q_{-1}	M(被乘数)	$[-x]_{尾补} = 00.100111$
1110011	0011011	0	1011001	10 -> sub (A-M)
0011010	0011011	0		shift
0001101	0001101	1		done

$$[x \cdot y]_{阶移尾补} = 01, 010; 00.011010001101$$

③ 规格化（左规，左移1位）

$$[x \cdot y]_{阶移尾补} = 1, 001; 00.110100011010$$

④ 舍入

$$[x \cdot y]_{阶移尾补} = 1, 001; 00.110100$$

$$x \cdot y = 0.110100 \times 2^{001}$$